

1. Title

Receipt and Invoice Digitizer



2. Introduction

In modern business environments, receipts and invoices are generated at high volumes across retail, logistics, services, and enterprise operations. These documents often exist in unstructured physical or semi-structured digital formats such as scanned images or PDFs. Manual handling of such documents leads to inefficiencies, human errors, loss of records, delayed reimbursements, and limited financial visibility.

The **Receipt & Invoice Digitizer** project aims to solve this problem by providing an automated, AI-assisted system that converts unstructured receipt and invoice documents into machine-readable digital data. The application is implemented as a **multi-page Streamlit web application**, integrating advanced image preprocessing techniques with **Google Gemini AI** for Optical Character Recognition (OCR) and structured information extraction.

This document describes **Milestone 1**, which focuses on building a **robust digitization foundation**—from document ingestion to OCR extraction and UI visualization—without yet emphasizing long-term analytics optimization or enterprise integrations.

3. Problem Statement

Organizations and individuals routinely process large volumes of receipts and invoices. The traditional workflow typically involves:

- Manual data entry into spreadsheets or accounting systems
- Storing physical copies for compliance
- High probability of transcription errors
- Inconsistent formatting across vendors
- Difficulty in tracking expenses over time

These challenges result in:

- Increased operational cost
- Reduced accuracy in financial records
- Delays in expense reconciliation
- Poor audit readiness

There is a strong need for an **automated, intelligent, and scalable solution** that can:

- Accept receipts and invoices in common formats
 - Extract text accurately
 - Convert unstructured content into structured data
 - Present results clearly to users
-

4. Milestone 1 Objective

The primary objective of **Milestone 1** is to design and implement a **robust, secure, and extensible document digitization foundation** capable of reliably converting physical receipts and invoices into structured digital data. This milestone focuses on establishing the **core technical pipeline** that will support all subsequent enhancements such as analytics, reporting, and long-term data persistence.

Milestone 1 is intentionally scoped to emphasize **correctness, reliability, and architectural soundness**, ensuring that downstream milestones can be built without refactoring core components. The system is designed to operate consistently across multiple document types while maintaining high OCR accuracy, predictable behavior, and a user-friendly interaction model.

Key Objectives

1. Multi-Format Document Ingestion

Enable secure and reliable ingestion of common receipt and invoice formats, including **JPG, PNG, and PDF files**. The system must correctly identify file types, validate file size and integrity, and handle both single-page and multi-page documents without manual intervention.

2. Standardized Image Conversion Pipeline

Convert all supported document formats into a **uniform, OCR-ready image representation**. PDFs must be safely rendered into individual page images, and all image outputs must be standardized (RGB format, consistent resolution) to ensure predictable downstream processing.

3. Automated Image Preprocessing for OCR Optimization

Implement an automated preprocessing layer that enhances OCR performance by applying operations such as **grayscale conversion, noise reduction, contrast enhancement, and binarization**. This preprocessing must run transparently in the background and be resilient to variations in document quality.

4. Single-Call OCR and Structured Data Extraction

Integrate **Google Gemini AI** to perform OCR and semantic understanding in a **single API call per document/page**. The system must extract not only raw text but also **structured bill data**, including vendor details, dates, line items, tax values, totals, currency, and payment method.

5. Strict Schema Enforcement and Data Normalization

Enforce a predefined JSON schema on all extracted data to ensure consistency and database

compatibility. Missing or ambiguous values must be handled using safe defaults, and numeric fields must be normalized to valid data types to prevent downstream failures.

6. Session State Management and Workflow Continuity

Maintain application state across Streamlit reruns using structured session state management. This ensures that uploaded files, processed images, extracted results, and user actions persist seamlessly during navigation, reducing redundant computation and improving user experience.

7. User-Centric and Intuitive Interface Design

Provide a **clean, minimal, and user-friendly interface** that clearly guides users through upload, processing, and result inspection. The UI must support image previews, extracted data visualization, and logical navigation without overwhelming the user.

8. Controlled Error Handling and Fault Tolerance

Implement comprehensive error handling across ingestion, preprocessing, OCR, and extraction stages. Failures must be **graceful, informative, and non-destructive**, ensuring that partial errors do not crash the application or corrupt session state.

9. Foundation for Persistent Storage and Analytics

Although advanced analytics and reporting are reserved for later milestones, Milestone 1 establishes the **data structures, schemas, and interfaces** required for seamless integration with persistent storage systems and analytical dashboards in future phases.

Outcome of Milestone 1

By the completion of Milestone 1, the system delivers a **production-ready core digitization pipeline** capable of transforming unstructured receipt and invoice documents into reliable, structured digital records. This milestone ensures technical stability, modularity, and scalability, forming a solid base for advanced features in subsequent milestones.

5. Milestone 2 Objective

The primary objective of **Milestone 2** is to extend the foundational digitization pipeline established in Milestone 1 into a **reliable, intelligent, and validation-driven data processing system** capable of producing high-quality, normalized, and persistent financial records from OCR-extracted content.

While Milestone 1 focuses on accurate document ingestion and OCR-based extraction, Milestone 2 emphasizes **data integrity, consistency, validation, and long-term usability**. This milestone ensures that extracted receipt and invoice data is not only readable, but also **trustworthy, comparable, and suitable for analytics, querying, and financial tracking**.

Milestone 2 is intentionally scoped to address real-world challenges such as inconsistent document formats, OCR inaccuracies, multi-currency transactions, duplicate uploads, and ambiguous tax structures, without requiring changes to the core ingestion or OCR architecture.

Key Objectives

1. Deterministic Field Extraction Using Regex and NLP

Augment AI-based OCR extraction with deterministic fallback mechanisms using regular expressions and lightweight NLP model. The system must identify and recover critical fields such

as invoice number, dates, totals, currency, tax values, and line items when AI extraction is incomplete or weak, without overriding confident AI outputs.

2. NLP-Based Vendor Name Identification

Implement NLP-based analysis to accurately identify vendor names from OCR text headers. The solution must account for positional importance, capitalization patterns, legal entity suffixes, and keyword filtering to distinguish vendor names from addresses, metadata, and transactional labels.

3. Structured Data Normalization and Standardization

Normalize all extracted fields into database-safe, query-ready formats. This includes enforcing consistent casing (uppercase text fields), standardized date and time formats, numeric type safety, length constraints, and default value handling to eliminate ambiguity and prevent downstream data corruption.

4. Multi-Currency Handling and USD Standardization

Enable support for receipts and invoices issued in multiple currencies. All monetary values must be normalized and converted into a single internal reporting currency (USD) while preserving original currency information and applied exchange rates to maintain transparency and auditability.

5. Robust Amount Validation Across Pricing Models

Implement a validation framework capable of safely handling both tax-inclusive and tax-exclusive pricing models. The system must validate subtotal, tax, and total relationships using tolerance-based logic to accommodate OCR rounding errors while detecting genuine inconsistencies in extracted financial data.

6. Logical Duplicate Detection Without Schema Changes

Prevent duplicate bill storage by introducing logical duplicate detection based on business attributes such as vendor name, invoice number, purchase date, and total amount. This detection must function correctly even when the same physical bill is uploaded from different files or scans, without requiring modifications to the database schema.

7. Persistent Relational Storage Using SQLite

Store validated and normalized receipt and invoice data in a lightweight, relational SQLite database. The storage layer must ensure referential integrity between bills and line items, support transaction-safe inserts, and provide indexed access for efficient querying.

8. Search-Ready and Analytics-Compatible Data Design

Prepare stored data for downstream search and analytical operations by enabling efficient querying based on vendor, date, payment method, and amount. Although advanced dashboards are addressed in later milestones, Milestone 2 ensures the database schema and stored data are analytics-ready.

9. Controlled Validation Feedback and User Awareness

Provide clear, non-disruptive validation feedback to users through warnings rather than hard failures wherever appropriate. The system must highlight potential data quality issues while preserving user control over data persistence decisions.

Outcome of Milestone 2

By the completion of Milestone 2, the system evolves from a document digitization pipeline into a reliable financial data processing platform. Extracted receipt and invoice data is deterministically recovered, normalized, validated, de-duplicated, and persistently stored in a structured database.

This milestone ensures that all stored records are accurate, consistent, and analytics-ready, establishing a solid foundation for advanced reporting, dashboards, and intelligent financial insights in subsequent milestones.

6. Milestone 3 Objective

The primary objective of Milestone 3 is to transform the structured, validated, and persistently stored financial data generated in Milestones 1 and 2 into meaningful analytical insights, interactive visualizations, and intelligent summaries.

While previous milestones focused on ingestion, extraction, validation, and storage, Milestone 3 introduces a full analytics layer capable of:

- Providing real-time financial visibility
- Enabling interactive dashboard-based exploration
- Generating automated statistical insights
- Delivering AI-generated natural language summaries
- Supporting export and reporting capabilities

This milestone completes the data lifecycle by enabling actionable intelligence derived from stored receipt and invoice data.

Key Objectives

1. Interactive Analytics Dashboard

Develop a fully interactive dashboard using Streamlit that enables users to visualize and explore financial data through:

- Key performance indicators (KPIs)
- Time-series spending trends
- Vendor-level spending breakdowns
- Payment method analysis
- Transaction frequency analysis
- Item-level spending analysis

The dashboard must update dynamically based on user-selected filters such as date range and payment method.

2. Structured Data Analytics Engine

Implement a modular analytics engine capable of computing statistical aggregations and derived datasets required for visualization and reporting.

The analytics engine must support:

- Monthly spending aggregation
- Transaction frequency analysis
- Vendor spending rankings
- Payment method distribution
- Tax and subtotal breakdowns
- Item-level aggregation

All analytics computations must operate efficiently using in-memory Pandas dataframes.

3. Advanced Data Visualization Layer

Implement a professional visualization layer using Plotly to render interactive charts, including:

- Line charts for spending trends
- Bar charts for transaction counts and vendor rankings
- Pie charts for vendor and payment distribution
- Histogram charts for transaction value analysis
- Cumulative spending curves
- Year-over-year comparison charts

Charts must support:

- Hover interactions
- Tooltips
- Dynamic filtering
- Responsive layout

4. Automated Statistical Insight Generation

Implement a statistical insight engine that converts numerical analytics into human-readable summaries.

This engine must automatically identify:

- Spending increases or decreases
- Highest spending vendors
- Most frequently purchased items
- Dominant payment methods
- Spending peaks and patterns

Insights must be generated dynamically based on filtered data.

5. AI-Generated Financial Insights using Gemini

Integrate Google Gemini AI to generate intelligent financial summaries and interpretations based on structured analytics data.

The AI insights engine must:

- Generate structured financial analysis
- Highlight anomalies and trends
- Identify behavioral patterns
- Provide business-level financial interpretation

AI generation must be deterministic, structured, and triggered only on user request.

6. Efficient Data Retrieval and Caching

Implement optimized data retrieval mechanisms to ensure fast dashboard performance.

This includes:

- Cached database queries
- Efficient dataframe transformations
- Lazy computation of expensive analytics
- Minimal redundant computation

Caching improves responsiveness and scalability.

7. Export and Reporting Capability

Enable export of financial data into standard reporting formats including:

- CSV export
- Excel export
- PDF export
- Detailed line-item export

Exports must support both summary and detailed views.

8. Modular and Scalable Dashboard Architecture

Ensure the dashboard architecture is modular, allowing independent development and extension of:

- Analytics logic
- Chart rendering
- Insight generation
- AI integration
- Export functionality

This design ensures long-term maintainability and scalability.

Outcome of Milestone 2

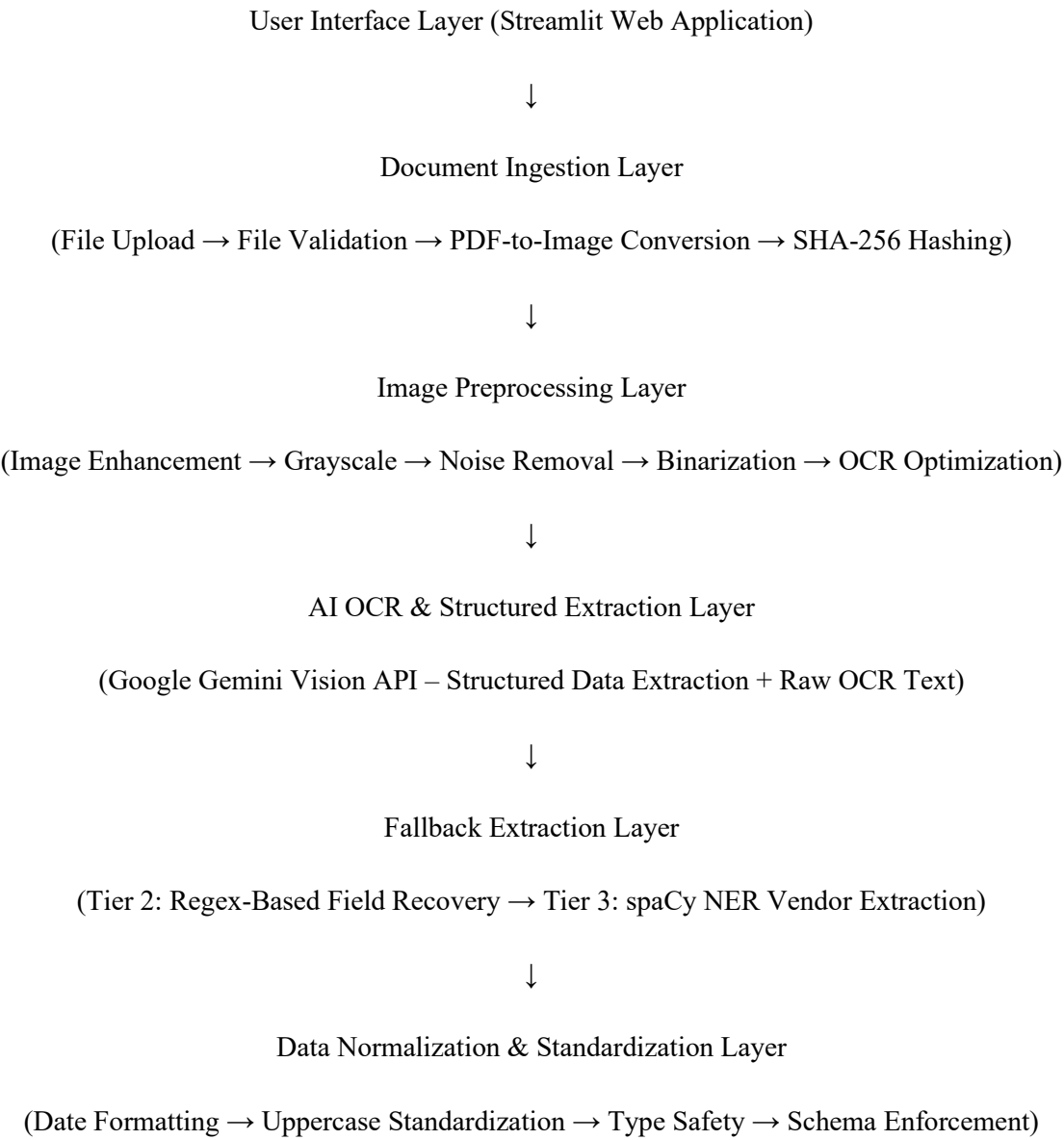
By the completion of Milestone 3, the system evolves from a data extraction and storage platform into a full-featured financial analytics system.

The system now provides:

- Interactive dashboard visualization
- Automated statistical insights
- AI-generated financial summaries
- Vendor and payment analysis
- Spending trend analysis
- Exportable financial reports

This milestone completes the intelligent analytics layer, enabling users to derive meaningful insights, track financial behavior, and make informed decisions based on receipt and invoice data.

7. High-Level Architecture





Currency Conversion Layer

(Multi-Currency Detection → Exchange Rate Application → USD Standardization)



Validation & Duplicate Detection Layer

(Amount Validation → Tax Consistency Checks → Logical Duplicate Detection)



Persistent Storage Layer

(SQLite Database – Bills Table + Line Items Table)



Data Retrieval Layer

(Database Queries → Cached Data Loading → Structured DataFrames)



Analytics Processing Layer

(KPI Computation → Monthly Trends → Vendor Analysis → Payment Analysis → Item Analysis)



Visualization Layer

(Interactive Charts using Plotly – Line, Bar, Pie, Histogram, YoY Analysis)



Statistical Insight Generation Layer

(Rule-Based Insight Engine → Automated Trend Detection → Text Insight Generation)



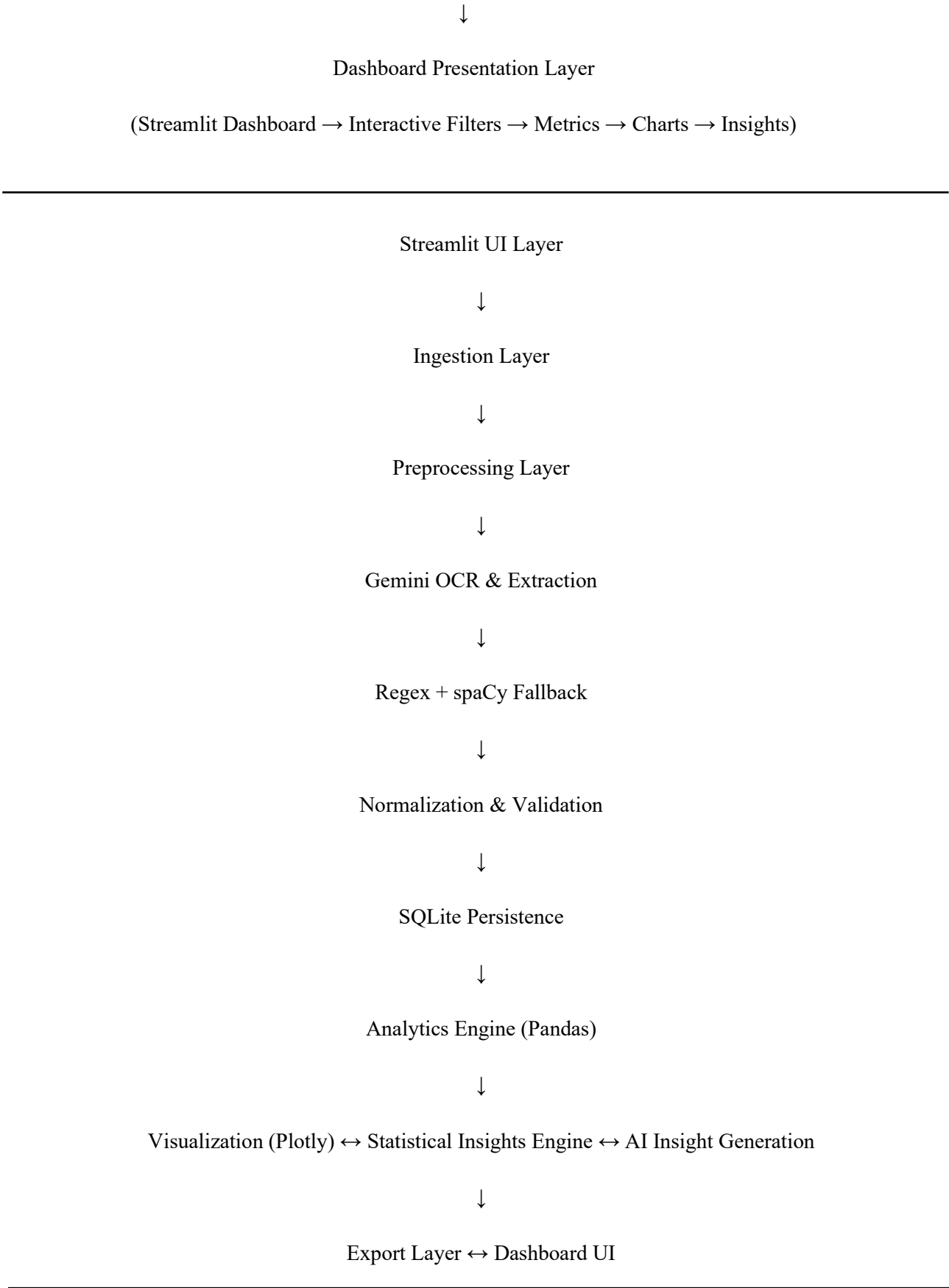
AI Insight Generation Layer

(Google Gemini AI → Financial Interpretation → Intelligent Summary Generation)



Export & Reporting Layer

(CSV Export → Excel Export → PDF Export → Detailed Line Item Reports)



8. Technology Stack

Layer	Technology	Purpose
User Interface (UI)	Streamlit	Multi-page interactive web application for document upload, dashboard visualization, analytics, and AI insights
Backend Language	Python 3.12+	Core programming language used for ingestion,

		processing, analytics, AI integration, and dashboard logic
OCR & AI Extraction	Google Gemini API (gemini-2.5-flash)	Performs OCR, structured receipt extraction, and AI-generated financial insights
Image Processing	OpenCV, Pillow (PIL)	Image preprocessing including grayscale conversion, binarization, noise removal, and contrast enhancement
PDF Handling	pdf2image, Poppler	Converts PDF documents into OCR-ready images
NLP & Regex Processing	spaCy (en_core_web_sm), Python re	Vendor name extraction using NER and deterministic fallback extraction using regex
Data Normalization & Validation	Custom Python modules	Standardizes extracted fields, enforces schema consistency, validates financial correctness
Currency Conversion	Custom Python module + Exchange Rate API / static rates	Converts multi-currency receipts into USD while preserving original currency metadata
Database & Persistence	SQLite	Relational database storing bills and line items with transactional integrity
Data Retrieval Layer	Custom database.py module	Provides structured APIs for querying bills, line items, and analytics data
Analytics Engine	Pandas	Performs statistical aggregation, KPI computation, and financial analytics
Visualization Engine	Plotly (plotly.graph_objects)	Builds interactive charts including line, bar, pie, histogram, and comparison charts
AI Insight Generation	Google Gemini API, hashlib, json	Generates intelligent financial summaries using structured analytics data
Dashboard & State Management	Streamlit Session State, Streamlit Cache	Maintains application state and improves performance using caching
Export & Reporting	Pandas, ReportLab, XLSXWriter, io.BytesIO	Enables export of financial data to CSV, Excel, and PDF formats
File Security & Hashing	hashlib (SHA-256)	Ensures file integrity and prevents duplicate processing
Date & Time Handling	datetime	Handles transaction date parsing and normalization
Data Serialization	JSON	Structured data exchange between modules and AI services
Dependency Management	pip, virtualenv / venv	Python package and environment management
Version Control	Git, GitHub	Source code management, version tracking, and collaboration
Development Environment	VS Code / PyCharm	Code development, debugging, and project management
Operating System Support	Windows, Linux, macOS	Cross-platform compatibility

9. Modules Implemented

8.1 Ingestion Layer (ingestion.py)

Purpose:

The ingestion layer acts as the **gateway** between user uploads and internal processing. Its responsibility is to **safely load documents**, normalize them, and produce a consistent internal representation.

Key Responsibilities

- Detect file type (image vs PDF)
- Validate file integrity
- Convert PDFs into page-wise images
- Apply security limits
- Generate file hash for change detection

Supported Inputs

- Local file paths
- Streamlit UploadedFile objects
- In-memory byte streams (BytesIO)

Security Controls

- Maximum PDF pages (prevents OOM attacks)
- Maximum image pixel threshold (prevents decompression bombs)
- SHA-256 file hashing

Outputs

- List[PIL.Image]
- Metadata dictionary containing:
 - Filename
 - File type
 - Number of pages
 - File hash
 - Truncation status

Design Principle

The ingestion layer never performs preprocessing or OCR. It only prepares data.

8.2 Preprocessing Layer (preprocessing.py)**Purpose:**

Raw images captured from cameras or scanners often contain noise, skew, low contrast, or transparency. The preprocessing layer ensures images are **OCR-ready**.

Processing Pipeline:

1. Safe image loading with EXIF correction
2. Transparency handling (RGBA → RGB with white background)
3. Grayscale conversion
4. Contrast enhancement
5. Otsu thresholding (binarization)

6. Noise removal (median filtering)
7. Optional resizing for performance optimization

Output:

Clean, binary PIL image optimized for OCR

Design Principle:

Preprocessing is **purely visual** and **content-agnostic**.

8.3 OCR & Extraction Layer (ocr.py)

Purpose:

This module performs OCR and semantic understanding using Google Gemini AI, extracting structured receipt and invoice data in a single API call.

Key Responsibilities

- Send preprocessed images to Gemini Vision API
- Enforce strict JSON-only structured output
- Extract both raw OCR text and structured bill data
- Handle AI failures and malformed responses gracefully

Extracted Data

- Raw OCR text (for fallback and auditing)
- Vendor name
- Invoice number
- Purchase date and time
- Currency
- Line items (description, quantity, pricing)
- Tax amount
- Total amount
- Payment method

Design Principle

Single-call extraction minimizes latency and cost while ensuring consistent structured output.

8.4 Regex & NLP Fallback Extraction Layer (regex_patterns.py, field_extractor.py, vendor_extractor_spacy.py)

Purpose

This layer provides deterministic and NLP-assisted fallback extraction when AI-generated structured output is incomplete, ambiguous, or unreliable. It ensures critical business fields are recovered before normalization and validation.

Key Responsibilities

- Detect weak, missing, or unreliable fields in AI extraction
- Extract structured fields using deterministic regex patterns
- Perform vendor name recovery using spaCy Named Entity Recognition (NER)
- Apply fallback logic selectively without overwriting strong AI-derived values

Techniques Used

- Regex-based pattern matching for deterministic field recovery:
 - Dates
 - Invoice / receipt numbers
 - Subtotal, tax, and total amounts
 - Currency detection
 - Payment methods
- **spaCy Named Entity Recognition (NER)** for vendor name extraction:
 - Uses ORG (Organization) entity detection
 - Applied only when vendor name remains weak after regex fallback
 - Operates on raw OCR text returned by Gemini
 - If the spaCy model is unavailable, the system degrades gracefully without blocking extraction.

Fallback Confirmation Logic

- A field is considered *weak* if it is:
 - None, empty, or whitespace
 - Zero-value for monetary fields
 - Invalid placeholder values (null, undefined)
- Regex and spaCy fallbacks are triggered only for weak fields
- Strong AI-extracted fields are preserved and never overwritten

Inputs

Raw OCR text extracted by Gemini AI

Outputs

Partially or fully recovered structured bill fields suitable for normalization

Design Principle

Fallback extraction is conservative, tiered, and non-destructive:

- Tier 1: Gemini AI structured output
- Tier 2: Regex-based deterministic recovery
- Tier 3: spaCy NER-based vendor identification

Fallback logic improves robustness without introducing hallucinations or uncontrolled overrides.

8.5 Data Normalization Layer (normalizer.py)

Purpose:

The normalization layer standardizes extracted data to ensure consistency, database compatibility, and reliable querying.

Key Responsibilities

- Normalize date and time formats (ISO-compliant)
- Convert all textual fields to uppercase
- Enforce length constraints for database fields
- Safely convert numeric values
- Apply default values for missing fields
- Normalize line item structures

Normalized Fields

- Vendor name
- Invoice number
- Currency code
- Payment method
- Line item descriptions
- Monetary values

Design Principle

Normalization guarantees that all downstream systems receive clean, predictable, and validated data formats.

8.6 Currency Conversion Layer (currency_converter.py)

Purpose:

This module converts all extracted monetary values into USD to enable unified analytics while preserving original currency information.

Key Responsibilities

- Detect non-USD currencies
- Apply exchange rates for conversion
- Convert subtotal, tax, total, and line item values
- Preserve original currency, original amounts, and exchange rate metadata

Outputs

- USD-normalized monetary values
- Original currency audit fields

Design Principle

Currency conversion enhances analytics uniformity without losing financial transparency.

8.7 Validation Layer (validation.py)

Purpose

The validation layer ensures numerical consistency and financial correctness of extracted bill data before it is persisted. It focuses strictly on amount validation.

Key Responsibilities

- Validate financial totals for logical consistency
- Handle OCR-induced rounding and precision errors using tolerance thresholds
- Detect inconsistent or suspicious monetary values
- Provide structured validation results for downstream decision-making

Validation Logic

The system evaluates extracted amounts using two accepted accounting models:

- Tax-Inclusive Model

$$\text{sum}(\text{items}) \approx \text{total_amount}$$

- Tax-Exclusive Model

$$\text{sum}(\text{items}) + \text{tax_amount} \approx \text{total_amount}$$

A bill is considered valid if either model passes within a configurable tolerance (± 0.02).

Tolerance Handling

- Small numeric deviations caused by OCR errors are tolerated
- Prevents false negatives due to rounding, formatting, or currency conversion noise

Outputs

```
{    "is_valid": bool,

    "items_sum": float,

    "tax_amount": float,

    "total_amount": float,

    "errors": [...],

    "warnings": [...] }
```


Design Principle

Validation is corrective and warning-driven, not destructive.

8.8 Logical Duplicate Detection Layer (duplicate.py)\

Purpose

This layer ensures that duplicate bills are not stored multiple times by applying logical matching rules after normalization and currency conversion.

Key Responsibilities

- Detect hard duplicates that must block saving
- Detect soft duplicates that require user awareness
- Provide clear reasoning for duplicate detection outcomes
- Operate independently of extraction logic

Duplicate Detection Strategy

- Hard Duplicate Detection (Blocking):

Matches on:

- Invoice number
- Vendor name (case-insensitive)
- Purchase date
- Total amount (within tolerance ± 0.02)

- Soft Duplicate Detection (Warning):

Applied when invoice number is missing

Matches on:

- Vendor name
- Purchase date
- Total amount (within tolerance)

Inputs

- Fully normalized bill data
- User ID (for future multi-user isolation)

Outputs

```
{ "duplicate": bool,
```

```
  "soft_duplicate": bool,
```

```
"reason": "Human-readable explanation" }
```

Design Principle

Duplicate detection is logical, deterministic, and isolated:

8.9 Database Persistence Layer (database.py)

Purpose:

Provides reliable, persistent storage of bills and line items using a relational database model.

Key Responsibilities

- Initialize database schema
- Insert bills and associated line items
- Enforce foreign key relationships
- Support cascade deletion
- Enable retrieval for history and analytics

Storage Model

- bills table (header-level data)
- lineitems table (item-level data)

Design Principle

Database operations are atomic, transactional, and integrity-safe.

Table: bills

Stores high-level information about each receipt or invoice.

Column Name	Data Type	Description
bill_id	INTEGER (Primary Key, Auto Increment)	Unique identifier for each bill
user_id	INTEGER (Default: 1)	User identifier (future multi-user support)
invoice_number	VARCHAR(100)	Invoice or receipt number
vendor_name	VARCHAR(255) NOT NULL	Name of the merchant or vendor
purchase_date	DATE NOT NULL	Date of transaction
purchase_time	TIME	Time of transaction (if available)
subtotal	DECIMAL(10,2)	Amount before tax
tax_amount	DECIMAL(10,2)	Tax applied to the bill
total_amount	DECIMAL(10,2)	Final payable amount
currency	VARCHAR(10)	Stored currency (USD after conversion)
original_currency	VARCHAR(10)	Currency detected from receipt
original_total_amount	DECIMAL(10,2)	Total amount in original currency
exchange_rate	DECIMAL(10,6)	Exchange rate used for conversion
payment_method	VARCHAR(50)	Payment mode (Cash, Card, UPI, etc.)
created_at	TIMESTAMP	Record creation timestamp

Table: lineitems

Stores individual item-level details linked to a bill.

Column Name	Data Type	Description
item_id	INTEGER (Primary Key, Auto Increment)	Unique identifier for each line item

bill_id	INTEGER NOT NULL	Reference to parent bill
description	TEXT	Item name or description
quantity	INTEGER	Quantity purchased
unit_price	DECIMAL(10,2)	Price per unit
total_price	DECIMAL(10,2)	Total price for the item

8.10 User Interface & Workflow Layer (app.py)

Purpose:

This module orchestrates the complete user workflow and presents extracted data through an intuitive interface.

Key Responsibilities

- Manage multi-page navigation
- Maintain session state across reruns
- Provide upload, processing, and save controls
- Display extracted data and validation feedback
- Support history browsing and analytics access

Design Principle

The UI layer is user-centric, stateful, and resilient to reruns and partial failures.

8.11 Dashboard UI Layer (dashboard_page.py)

Purpose:

Provides the main Streamlit dashboard interface and orchestrates analytics, charts, insights, and AI features.

Key Responsibilities:

- Load bills and line items from database
- Apply filters (date range, payment method)
- Compute KPIs
- Render charts and visualizations
- Display statistical insights
- Trigger AI insights generation
- Provide export functionality

Key Functions:

- `page_dashboard()` — Main dashboard entry point
- `_cached_bills()` — Cached loader for bill records
- `_cached_items()` — Cached loader for line items
- `_render_ai_insights()` — Render AI insights output

Dependencies:

- analytics.py
- charts.py
- insights.py
- ai_insights.py
- exports.py
- database.py

8.12 Analytics Layer (analytics.py)

Purpose:

Provides pure data processing and aggregation logic.

Key Responsibilities:

- Clean and normalize dataframe structure
- Compute KPIs and aggregated metrics
- Prepare vendor analytics
- Prepare payment analytics
- Prepare item analytics

Key Functions:

- prepare_bills_dataframe()
- calculate_kpis()
- monthly_spending()
- monthly_transaction_counts()
- top_vendors()
- payment_distribution()
- high_value_transactions()
- prepare_items_dataframe()

Dependencies:

- pandas

Design Principle:

Pure computation layer with no UI or visualization logic.

8.13 Visualization Layer (charts.py)

Purpose:

Builds interactive Plotly charts for dashboard visualization.

Key Responsibilities:

- Generate line charts
- Generate bar charts
- Generate pie charts
- Generate histogram charts

- Generate comparison charts

Key Functions:

- `monthly_spending_line()`
- `vendor_bar_chart()`
- `payment_method_pie()`
- `transaction_histogram()`
- `top_items_bar()`
- `yoy_comparison()`

Dependencies:

- `plotly.graph_objects`

Design Principle:

Chart rendering is isolated from data processing logic.

8.14 Statistical Insights Layer (`insights.py`)

Purpose:

Generates human-readable insights from statistical analytics.

Key Responsibilities:

- Interpret numerical analytics
- Generate meaningful textual insights
- Highlight trends and anomalies

Key Functions:

- `monthly_spending_insight()`
- `vendor_insight()`
- `payment_insight()`
- `transaction_histogram_insight()`
- `top_items_insight()`

Dependencies:

- `pandas`

Design Principle:

Transforms analytics into user-friendly explanations.

8.15 AI Insight Generation Layer (`ai_insights.py`)

Purpose:

Uses Google Gemini AI to generate intelligent financial analysis.

Key Responsibilities:

- Build structured summary from analytics
- Generate AI insights using Gemini API
- Cache results using summary hash
- Enforce structured AI output format

Key Functions:

- build_summary()
- summary_hash()
- generate_ai_insights()

Dependencies:

- google.genai
- pandas
- hashlib
- json

Design Principle:

AI provides higher-level interpretation beyond statistical insights.

8.16 Export Layer (exports.py)

Purpose:

Provides data export capabilities.

Key Responsibilities:

- Export data to CSV
- Export data to Excel
- Export data to PDF
- Export detailed bill and line-item data

Key Functions:

- export_csv()
- export_excel()
- export_pdf()
- export_detailed_csv()
- export_detailed_excel()
- export_detailed_pdf()

Dependencies:

- pandas
- reportlab
- xlsxwriter

Design Principle:

Supports reporting and external data sharing.

9 Streamlit Application Design

9.1 Session State Management

Streamlit reruns the entire script on every interaction.

Session state is used to preserve:

- Uploaded file context
- API key
- Preprocessed images
- OCR results
- Navigation state
- Save status

This prevents:

- Redundant OCR calls
- Data loss on reruns
- UI inconsistency

9.2 Sidebar Navigation

The sidebar manages:

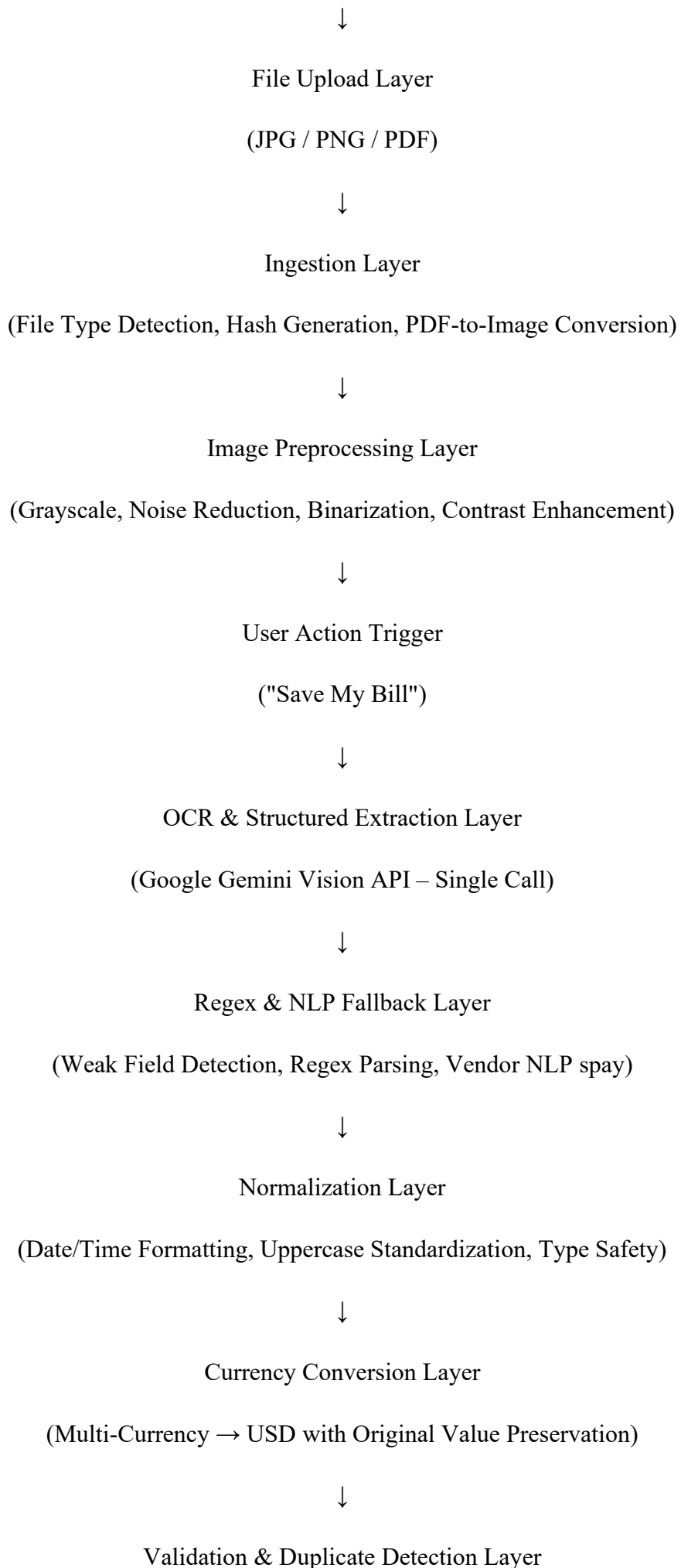
- **Gemini API key input**
- **Page navigation**
- **Application metadata**

The API key is stored only in session memory and never logged or persisted.

9.3 Upload & Process Workflow

Supported Scenarios:

- Single image receipts
- Single-page PDFs
- Multi-page PDFs (page-wise processing)



(Amount Consistency Checks, Logical Duplicate Detection)



Persistent Storage Layer

(SQLite Database – Bills & Line Items)



Presentation & Analytics Layer

(Streamlit Tables, History View, Dashboard Metrics)

9.4 Results Display

Results are presented using:

- Uploaded image
- Preprocessed image previews
- Structured bill data
- File metadata

Tabbed layout improves readability and user experience.

The screenshot displays the 'Digitizer' application interface, which is a receipt and invoice digitizer. The interface is divided into two main sections: '1. Input' and '2. Results'.

1. Input: This section contains a 'Select File' button, a 'Drag and drop file here' area, and a 'Browse files' button. A file named '081.jpg' (256.2KB) is shown as uploaded. A green status bar indicates 'Image loaded (Single page)'.

2. Results: This section is divided into two tabs: 'Results' and 'Metadata'. The 'Results' tab is active, showing 'Extracted Bill Information' and 'Persistent Storage'.

Extracted Bill Information: A table showing the extracted bill data.

id	invoice_number	vendor_name	purchase_date	purchase_time	payment_method	total_amount
12	1057981	RESTORAN WAN SHENG	2018-03-23	13 hours	CASH	\$6.70
11	1049519	RESTORAN WAN SHENG	2018-03-17	15 hours	CASH	\$9.60

Detailed Bill Items: A section for viewing detailed bill items. A dropdown menu shows 'Bill #11 - RESTORAN WAN SHENG - 2018-03-17'. Below it is a table showing the detailed items.

s_no	item_name	quantity	unit_price	item_total
1	Kopi (B)	1	2.2	2.2
2	Teh (B)	2	2.2	4.4

The 'Persistent Storage' section shows a table with the same data as the 'Extracted Bill Information' table.

The 'Metadata' tab is also visible, showing a table with the same data as the 'Extracted Bill Information' table.

The 'Navigation' section on the left includes links to 'Dashboard' and 'Upload & Process'.

<<

Digitizer

Receipt & Invoice Digitizer

API Configuration

Enter Gemini API Key

.....

👁

✓ API Key Loaded

Navigation

Dashboard

Upload & Process

History

Deploy

⋮

required to make necessary adjustments to its input tax claims, on the basis of the adjustments shown in this Tax Invoice / Adjustment Note

CUSTOMER'S COPY

Preprocessed Image

🖼 View Uploaded Image

💾 Save My Bill

☐ Extracting and saving bill...

✓ Amount validation passed

⚠ Duplicate bill detected. Reason: Invoice #7820F713 from GARDENIA BAKERIES (KL) SDN BHD on 2017-08-20 already exists. Bill not saved.



Financial Dashboard

Comprehensive insights into your spending patterns



Key Metrics



Smart Filters

Quick Select

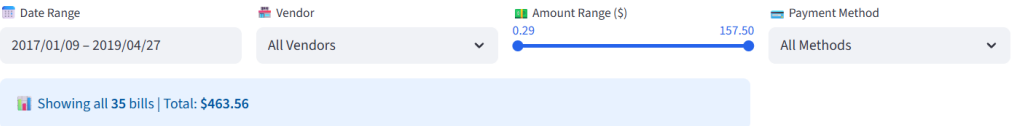
Custom Range



Smart Filters

Quick Select

Custom Range



Insights & Trends

[Overview](#) [Vendors & Payments](#) [Spending Patterns](#) [Item Insights](#)

Monthly Spending Trend



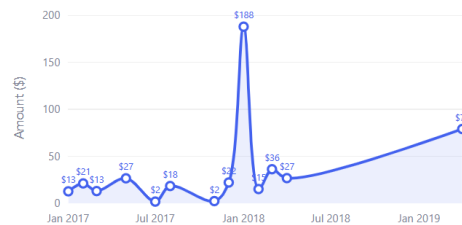
Number of Bills per Month



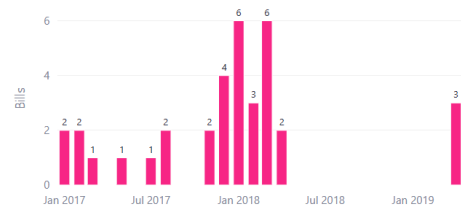
Insights & Trends

[Overview](#) [Vendors & Payments](#) [Spending Patterns](#) [Item Insights](#)

Monthly Spending Trend



Number of Bills per Month



Total spending by month. Look for peaks to spot high-cost periods.

Insight: Highest spending month: 2018-01 at \$187.96. Last month is up 194.9% vs prior month.

How many bills you had each month.

Insight: Busiest month: 2018-01 with 6 bills. Average volume is 2.7 bills per month.

10. Error Handling & Validation

The system implements structured error handling and validation to ensure data reliability, system stability, and user trust throughout the digitization workflow.

10.1 Upload-Level Validation

- Supports only JPG, PNG, and PDF formats
- Enforces maximum file size limits
- Detects unchanged re-uploads using file hashing
- Prevents invalid files from entering the pipeline

10.2 Ingestion & Preprocessing Safety

- Handles corrupted or partially readable files gracefully
- Limits PDF pages to prevent memory issues
- Falls back to original images if preprocessing fails

10.3 OCR & AI Extraction Handling

- Enforces JSON-only output from Gemini AI
- Detects malformed or incomplete AI responses
- Requests raw OCR text for fallback extraction

10.4 Regex & NLP Fallback Recovery

- Identifies weak or missing fields
- Applies regex-based extraction for dates, totals, tax, currency
- Uses NLP model spacy for vendor name detection
- Overrides only weak fields, preserving strong AI results

10.5 Data Normalization Controls

- Converts all values to safe, consistent formats
- Standardizes text to uppercase
- Applies default values for missing fields
- Prevents type and schema errors before storage

10.6 Currency Conversion Validation

- Converts non-USD currencies safely
- Preserves original currency and exchange rate
- Skips conversion if unsupported without failing

10.7 Amount Validation

- Supports both tax-inclusive and tax-exclusive bills
- Allows tolerance for OCR rounding errors
- Displays warnings instead of blocking user actions

10.8 Duplicate Detection

- Detects duplicates using invoice number, vendor, date, and amount
- Prevents accidental double storage of bills

10.9 Database Safety

- Uses transaction-based inserts
- Rolls back on failures
- Maintains referential integrity with foreign keys

10.10 User Feedback

- Clear warnings and error messages
- No application crashes on failure
- User remains in control of final actions

10.11 Dashboard Data Retrieval and Analytics Validation

- Validates database query results before performing analytics computations
- Handles empty datasets gracefully without causing runtime failures
- Prevents division-by-zero and invalid aggregation errors
- Automatically handles missing, null, or incomplete values in analytics processing
- Ensures safe dataframe transformations and aggregation using Pandas

10.12 Visualization Layer Safety

- Validates required fields before generating charts
- Prevents chart rendering when insufficient or invalid data is present
- Displays empty chart placeholders instead of crashing the dashboard
- Ensures safe rendering of interactive Plotly visualizations
- Handles chart generation errors without affecting other dashboard components

10.13 AI Insight Generation Validation

- Validates summary data before sending requests to Gemini AI
- Prevents AI calls when insufficient analytics data is available
- Verifies API key presence before initiating AI insight generation
- Handles API failures, timeouts, or malformed responses gracefully
- Prevents duplicate or redundant AI calls using summary hash comparison
- Displays structured error messages without interrupting dashboard functionality

10.14 Export and Reporting Safety

- Validates data availability before generating export files
- Prevents exporting empty or invalid datasets
- Ensures safe conversion of data into CSV, Excel, and PDF formats
- Uses memory-safe file generation through in-memory streams
- Handles export failures without affecting dashboard operation

10.15 Session State and Cache Validation

- Ensures safe initialization and access of Streamlit session state variables
 - Prevents invalid or stale session state data from affecting application behavior
 - Automatically refreshes cached data when database changes occur
 - Prevents redundant database queries through controlled caching
 - Maintains consistency between dashboard display and stored data
-

11. Project Work Timeline – Milestone 1 (14 Days)

Day	Date	Work Description
Day 1	29 Dec 2025	Project initialization, GitHub repository setup, initial codebase creation
Day 2	02 Jan 2026	Project folder structure setup, .gitignore creation, initial Streamlit app configuration
Day 3	03 Jan 2026	Dashboard UI development, frontend cleanup, tab naming refinements
Day 4	05 Jan 2026	Image preprocessing pipeline implementation (grayscale conversion, binarization, noise removal)
Day 5	06 Jan 2026	Gemini API key handling, PDF upload support, ingestion layer enhancements
Day 6	07 Jan 2026	Bug fixes, file handling corrections, ingestion and preprocessing stability improvements
Day 7	08 Jan 2026	OCR logic refinement, preprocessing performance optimization
Day 8	09 Jan 2026	OCR optimization review, preprocessing tuning, initial documentation drafting
Day 9	10 Jan 2026	SQLite database integration, UI–database linkage, structured data handling
Day 10	10 Jan 2026	Database schema refinement (bills & line items tables, keys, relationships)
Day 11	12 Jan 2026	Code refactoring, validation logic improvements, feature polishing
Day 12	12 Jan 2026	Formal documentation creation and organization (Milestone 1 completion)
Day 13	13 Jan 2026	README updates, documentation refinement, repository cleanup
Day 14	13 Jan 2026	Final documentation restructuring and Milestone 1 closure
Day 15	14 Jan 2026	Milestone 2 initialization, folder structure setup for extraction and validation modules
Day 16	15 Jan 2026	Regex pattern design for dates, invoice numbers, currency, tax, totals, and line items
Day 17	16 Jan 2026	Implementation of field_extractor.py using regex-based deterministic extraction
Day 18	17 Jan 2026	Normalization module implementation (uppercase normalization, numeric safety, date/time standardization)
Day 19	18 Jan 2026	Validation logic enhancement: tax-inclusive & tax-exclusive safe validation model
Day 20	19 Jan 2026	Duplicate detection logic based on invoice number, vendor, date, and total amount
Day 21	20 Jan 2026	Regex fallback integration using raw OCR text before normalization
Day 22	21 Jan 2026	NLP-based vendor name extraction using spaCy
Day 23	21 Jan 2026	Currency normalization and automatic conversion of non-USD totals to USD
Day 24	22 Jan 2026	spaCy NER integration, PPT and documentation updates
Day 25	23 Jan 2026	Milestone 3 initialization, dashboard module folder structure setup
Day 26	24 Jan 2026	Implementation of analytics.py for KPI computation and financial aggregation
Day 27	25 Jan 2026	Implementation of monthly spending, transaction trends, and vendor analytics functions
Day 28	26 Jan 2026	Implementation of charts.py for Plotly-based visualization (line, bar, pie charts)
Day 29	27 Jan 2026	Integration of charts into Streamlit dashboard interface
Day 30	28 Jan 2026	Implementation of dashboard_page.py main dashboard layout and UI structure
Day 31	29 Jan 2026	Implementation of dashboard filters (date range, payment method filtering)
Day 32	30 Jan 2026	KPI cards implementation (total spend, transactions, vendor count, averages)
Day 33	31 Jan 2026	Implementation of vendor analysis charts and payment distribution charts
Day 34	01 Feb 2026	Implementation of item-level analytics and item spend aggregation
Day 35	02 Feb 2026	Implementation of insights.py for automated statistical insight generation
Day 36	03 Feb 2026	Integration of automated insight text into dashboard UI
Day 37	04 Feb 2026	Implementation of ai_insights.py and Gemini integration for AI-generated financial summaries
Day 38	05 Feb 2026	AI insight prompt engineering, structured output enforcement, and testing

Day 39	06 Feb 2026	Implementation of caching mechanisms for database queries and analytics performance optimization
Day 40	07 Feb 2026	Implementation of export functionality (CSV, Excel export using Pandas and XLSXWriter)
Day 41	08 Feb 2026	Implementation of PDF export using ReportLab
Day 42	09 Feb 2026	Dashboard performance optimization, bug fixes, and error handling improvements
Day 43	10 Feb 2026	UI/UX refinement, dashboard layout improvements, and responsiveness enhancements
Day 44	11 Feb 2026	Milestone 3 documentation drafting, architecture updates, and module documentation
Day 45	12 Feb 2026	Final documentation completion, testing, validation, and Milestone 3 closure

12. Conclusion

The successful completion of Milestones 1, 2, and 3 establishes a comprehensive, intelligent, and production-ready Receipt & Invoice Digitizer system capable of managing the complete lifecycle of financial document processing and analysis.

The system now supports full end-to-end functionality, beginning with secure document ingestion and image preprocessing, followed by AI-driven OCR and structured data extraction using Google Gemini. Deterministic regex fallback and spaCy-based Named Entity Recognition ensure robust field recovery and vendor identification, while normalization, currency conversion, validation, and logical duplicate detection guarantee data accuracy, consistency, and integrity. Persistent storage using a relational SQLite database provides reliable and query-efficient long-term data management.

Milestone 3 extends the platform into a full analytics and financial intelligence system by introducing an interactive Streamlit dashboard, real-time KPI computation, and advanced data visualization using Plotly. Users can now analyze spending trends, vendor distributions, payment behavior, and item-level insights through dynamic and interactive charts. Automated statistical insight generation provides immediate interpretability of financial data, while integration with Google Gemini AI enables intelligent, structured financial summaries and higher-level analytical interpretation.

Additionally, the system now supports comprehensive reporting and export capabilities, allowing users to export financial data in CSV, Excel, and PDF formats for external analysis, auditing, and record-keeping. Performance optimization through caching and modular design ensures efficient data processing, scalability, and responsiveness even as data volume grows.

The modular and layered architecture ensures clear separation of concerns across ingestion, extraction, validation, storage, analytics, visualization, and AI interpretation layers. This design enables high maintainability, extensibility, and scalability, allowing future enhancements such as multi-user support, cloud deployment, real-time synchronization, automated categorization, and predictive financial analytics.

With a robust extraction pipeline, validated and structured financial database, interactive analytics dashboard, and AI-powered insight generation, the Receipt & Invoice Digitizer system has evolved into a complete financial document intelligence platform capable of transforming unstructured receipt and invoice data into actionable financial insights.